

Introduction to Scientific Computing

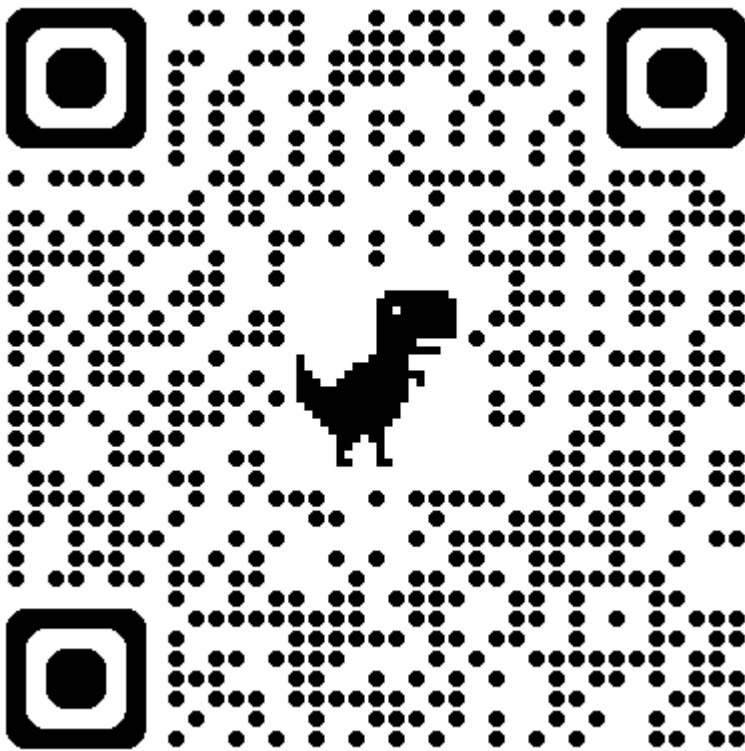
ENGR1050 – Lecture 24

Fall 2025

Nat Trask

University of Pennsylvania

Deep Learning with MLPs





Review

Backprop
Gradient descent
Polynomial fitting

Deep Learning

Learn to use deep neural
networks to fit functions to
engineering data

Review: Supervised learning

Goal: Given data points (x_i, y_i) , find parameters θ that best fit a model

The Problem

- Model: $\hat{y} = f(x; \theta)$ (prediction)
- Data: $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ (observations)
- Loss function: $L(\theta) = \sum_{i=1}^n (y_i - f(x_i; \theta))^2$ (error)

Optimization Approach:

- 1 Minimize loss: find θ^* where $\frac{\partial L}{\partial \theta} = 0$
- 2 **This is root finding!** Apply Newton's method
- 3 Or use optimization form: $\theta_{n+1} = \theta_n - \frac{\partial L / \partial \theta}{\partial^2 L / \partial \theta^2}$

Connection to Engineering:

- Fit helicopter control model to flight data
- Calibrate COVID model (β, γ) to case counts
- Train neural networks (gradient descent is simplified Newton)

Review: gradient descent

Recall the update for Newton's method:

$$x_{n+1} = x_n - \underbrace{\frac{1}{f''(x_n)}}_{\text{step distance}} \underbrace{f'(x_n)}_{\text{step direction}}$$

Computing second derivatives is expensive; if we have n variables, it costs n^2 to compute the gradient of every gradient.

To make this efficient, gradient descent introduces a fixed step size α :

$$x_{n+1} = x_n - \alpha f'(x_n) \leftarrow \text{Compute with automatic differentiation!}$$

Step size. α must be tuned to ensure convergence. Too large can overshoot; too small leads to slow progress.

Review: computing gradients with backprop

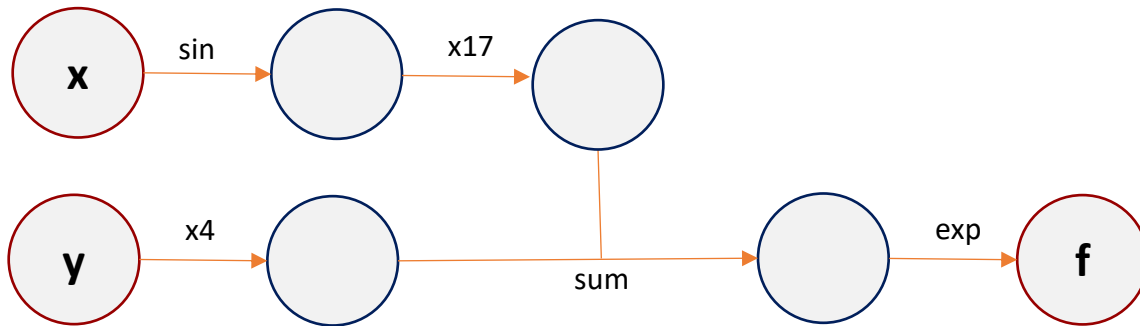
```
import torch
```

```
x = torch.tensor(3.0, requires_grad=True)
y = torch.tensor(4.0, requires_grad=True)
f = torch.exp(17 * torch.sin(x) + 4*y)
```



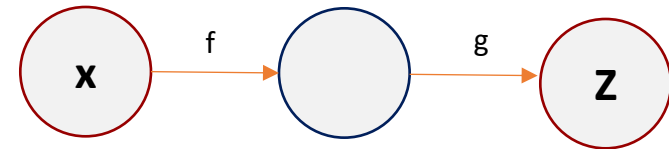
0.0s

Python



Step 1. Torch decomposes functions into a *computational graph*

$$z = f(g(x))$$



$$\frac{dz}{dx} = f'(g(x)) \cdot g'(x)$$

Step 2. Traversing the graph backward with the chain rule gives gradients

Review: Four step cookbook to take a gradient

Step 1: Define variables, set requires_grad=True

```
x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True) # A vector  
y = torch.tensor(3, requires_grad=True) # A scalar
```

Step 2: Evaluate forward pass of function to differentiate

```
f = x**2 + y * x # Some function of x and y
```

Step 3: Evaluate backward() on output to compute gradients

```
f.backward() # Compute gradients
```

Step 4: Access dfdx and dfdy gradients via .grad attribute

```
print(x.grad) # Gradient of f w.r.t. x  
print(y.grad) # Gradient of f w.r.t. y
```

Warning: Need to run `x.grad.zero_()` before next `backward()`

How did we do this for polynomial fitting? Forward pass

```
class modelClass:
    def __init__(self, a0, b0):
        # Initialize a linear model  $y = a*x + b$ 
        # a0 - (float) initial value for slope
        # b0 - (float) initial value for intercept

        self.a = torch.tensor(a0, requires_grad=True)
        self.b = torch.tensor(b0, requires_grad=True)

    def forward(self, x):
        # Given input x (assumed to be a scalar shape torch tensor), return predicted  $y = a*x+b$ 
        return self.a * x + self.b
```

How did we do this for polynomial fitting? Backward pass

```
# Initialize a linear model with a = b = 0
model = modelClass(a0=0.0, b0=0.0)

# Set up loop to train model with gradient descent
learning_rate = 0.0001
num_steps = 200
for step in range(num_steps):

    # Step 1: Take forward pass by evaluating model on each data point and computing the total loss
    total_loss = 0.0
    for i in range(len(x_data)):
        x = x_data[i]
        y_true = y_data[i]

        # Forward pass: compute predicted y
        y_pred = model.forward(x)

        # Compute loss (mean squared error)
        loss = (y_pred - y_true)**2
        total_loss += loss.item()

    # Step 2: Backward pass: compute gradients using automatic differentiation
    loss.backward()

    # Step 3: Update parameters using gradient descent
    with torch.no_grad():
        model.a -= learning_rate * model.a.grad
        model.b -= learning_rate * model.b.grad

    # Zero gradients after updating - otherwise they will accumulate
    model.a.grad.zero_()
    model.b.grad.zero_()
```

Step 1. Use model to evaluate loss

Step 2. Take gradient of loss with respect to model parameters

Step 3. Use gradient to update parameters



Review

Backprop

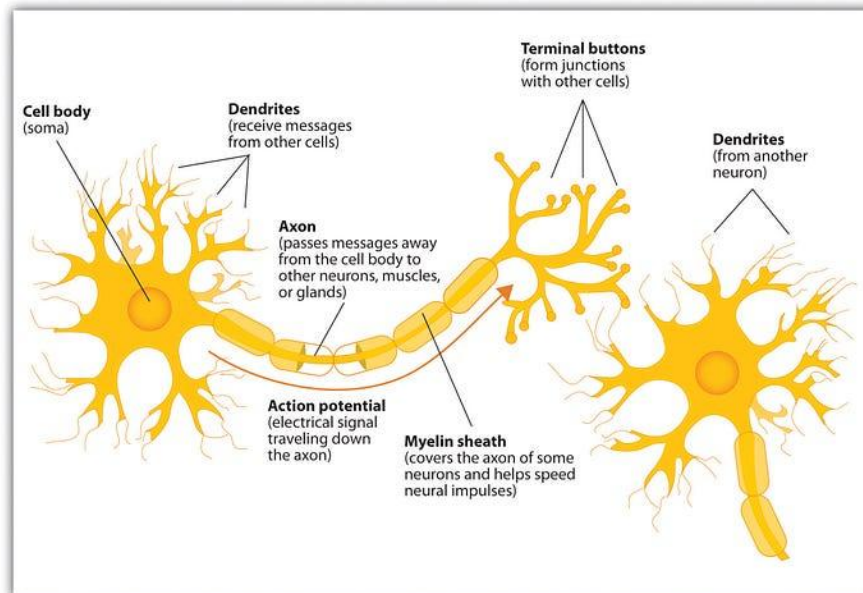
Gradient descent

Polynomial fitting

Deep Learning

**Learn to use deep neural
networks to fit functions to
engineering data**

Multilayer perceptrons (MLPs) are mathematical abstractions of neurons



Theorem (Cybenko)

Let σ be any continuous discriminatory function.

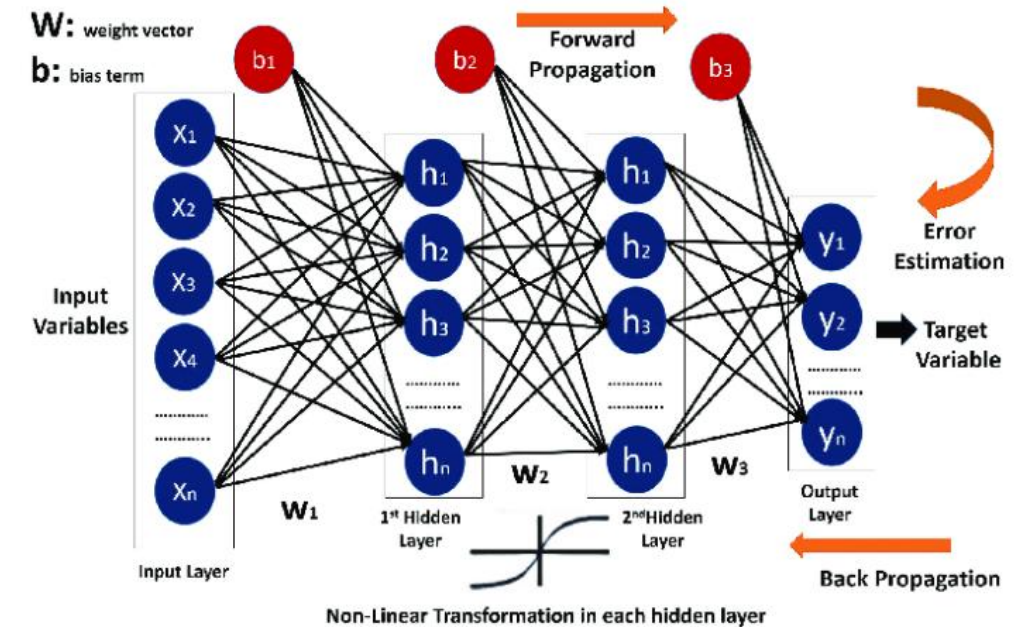
Then finite sums of the form

$$G(x) = \sum_{j=1}^N \alpha_j \sigma(w_j^T x + b_j), \text{ where } w_j \in \mathbb{R}^n, \alpha_j, b_j \in \mathbb{R}$$

are dense in $C(I_n)$.

In other words, given any $\varepsilon > 0$ and $f \in C(I_n)$, there is a sum $G(x)$ of the above form such that

$$|G(x) - f(x)| < \varepsilon, \quad \forall x \in I_n$$



$$\mathbf{h}_1 = \sigma(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1)$$

$$\mathbf{h}_2 = \sigma(\mathbf{W}_2 \mathbf{h}_1 + \mathbf{b}_2)$$

$\vdots$

$$\mathbf{h}_N = \sigma(\mathbf{W}_N \mathbf{h}_{N-1} + \mathbf{b}_N)$$

$$\mathbf{y} = \mathbf{W}_{out} \mathbf{h}_N$$

From polynomials to deep neural networks

$$y = W_1 + W_2x$$

```
class modelClass:
    def __init__(self, a0, b0):
        # Initialize a linear model y = a*x + b
        # a0 - (float) initial value for slope
        # b0 - (float) initial value for intercept

        self.a = torch.tensor(a0, requires_grad=True)
        self.b = torch.tensor(b0, requires_grad=True)

    def forward(self, x):
        # Given input x (assumed to be a scalar shape torch tensor), return predicted y = a*x+b
        return self.a * x + self.b
```

$$\begin{aligned} \mathbf{h}_1 &= \sigma(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) \\ \mathbf{h}_2 &= \sigma(\mathbf{W}_2\mathbf{h}_1 + \mathbf{b}_2) \\ &\vdots \\ \mathbf{h}_N &= \sigma(\mathbf{W}_N\mathbf{h}_{N-1} + \mathbf{b}_N) \\ \mathbf{y} &= \mathbf{W}_{out}\mathbf{h}_N \end{aligned}$$

```
class MLP_modelClass:
    def __init__(self, input_dim, hidden_dim, output_dim):
        # Initialize a 2-layer MLP
        # input_dim - dimension of input (e.g., 1 for scalar input)
        # hidden_dim - number of neurons in hidden layer
        # output_dim - dimension of output (e.g., 1 for scalar output)

        # Layer 1: input -> hidden
        self.W1 = torch.randn(hidden_dim, input_dim, requires_grad=True)
        self.b1 = torch.randn(hidden_dim, requires_grad=True)

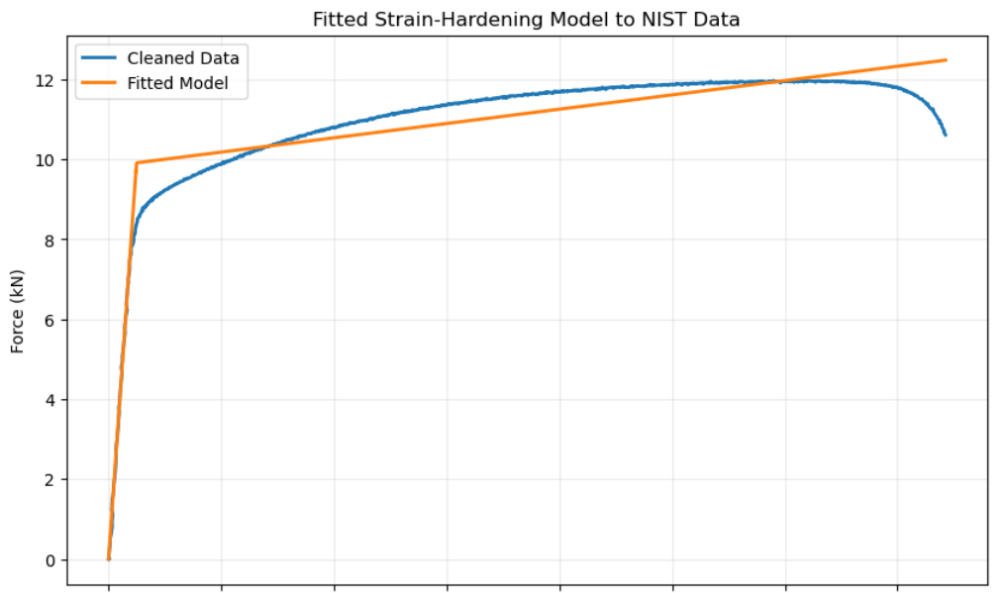
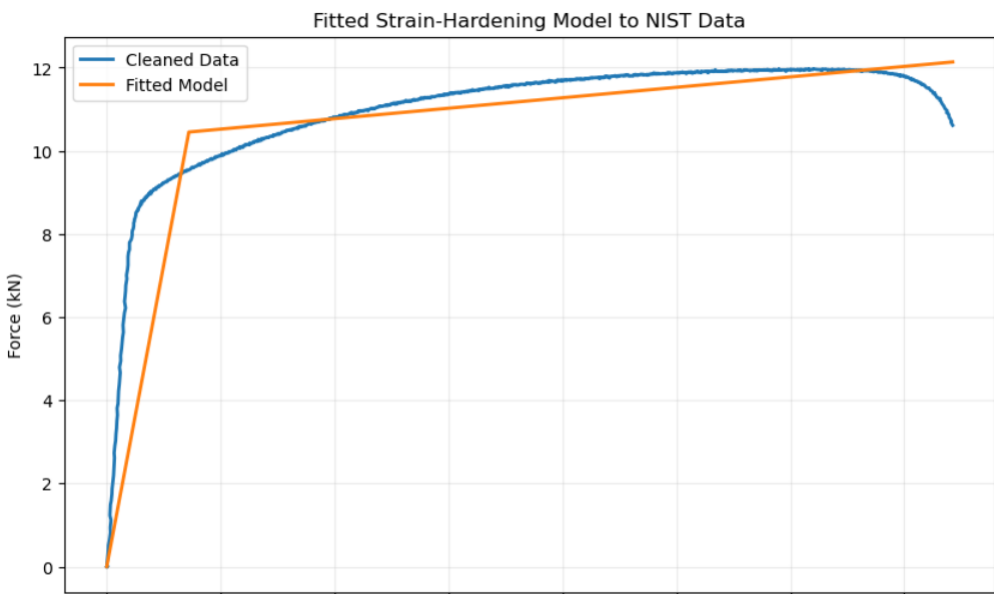
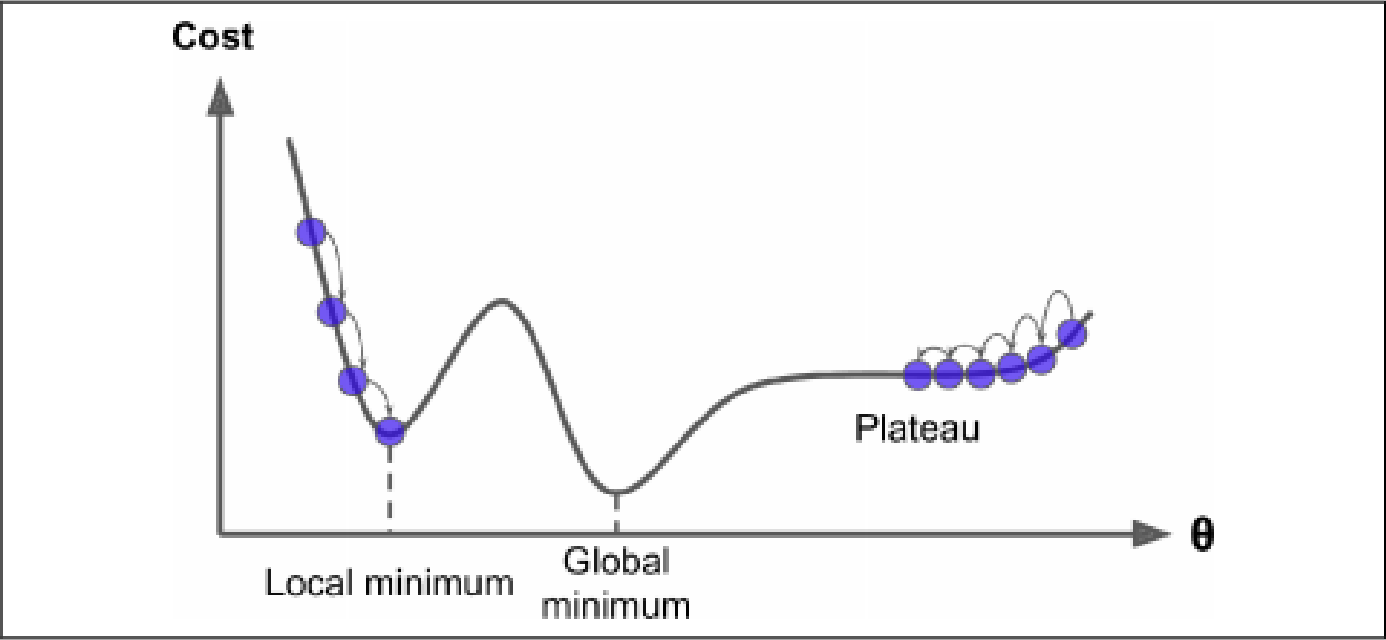
        # Layer 2: hidden -> output
        self.W2 = torch.randn(output_dim, hidden_dim, requires_grad=True)
        self.b2 = torch.randn(output_dim, requires_grad=True)

    def forward(self, x):
        # Layer 1: compute h1 = tanh(W1*x + b1)
        z1 = self.W1 @ x + self.b1    # Linear transformation
        h1 = torch.tanh(z1)           # Activation function

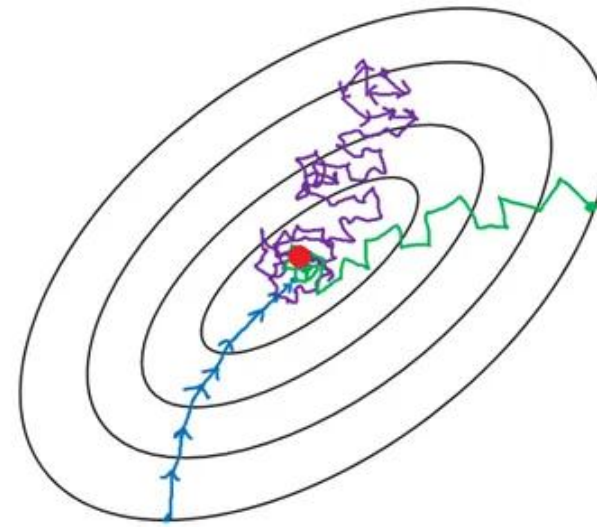
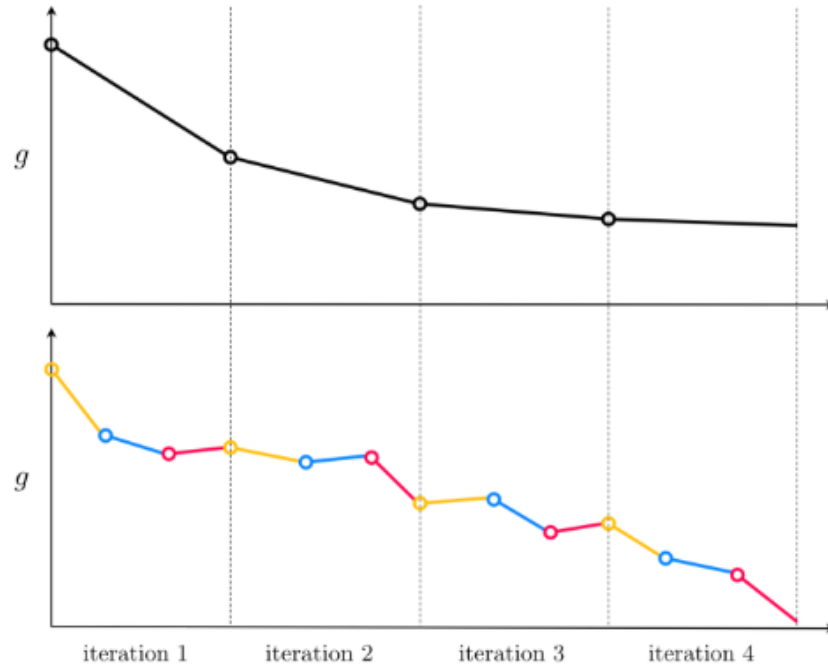
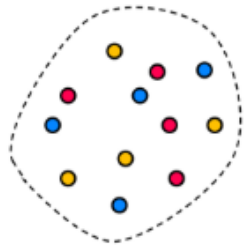
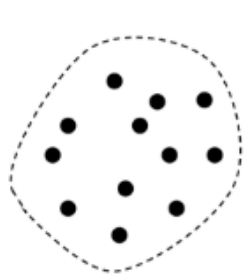
        # Layer 2: compute y = W2*h1 + b2
        y = self.W2 @ h1 + self.b2    # Linear transformation (no activation on output)

        return y
```

Training DNNs - Suboptimal local minima



Introducing minibatching (AKA *stochastic gradient descent*)

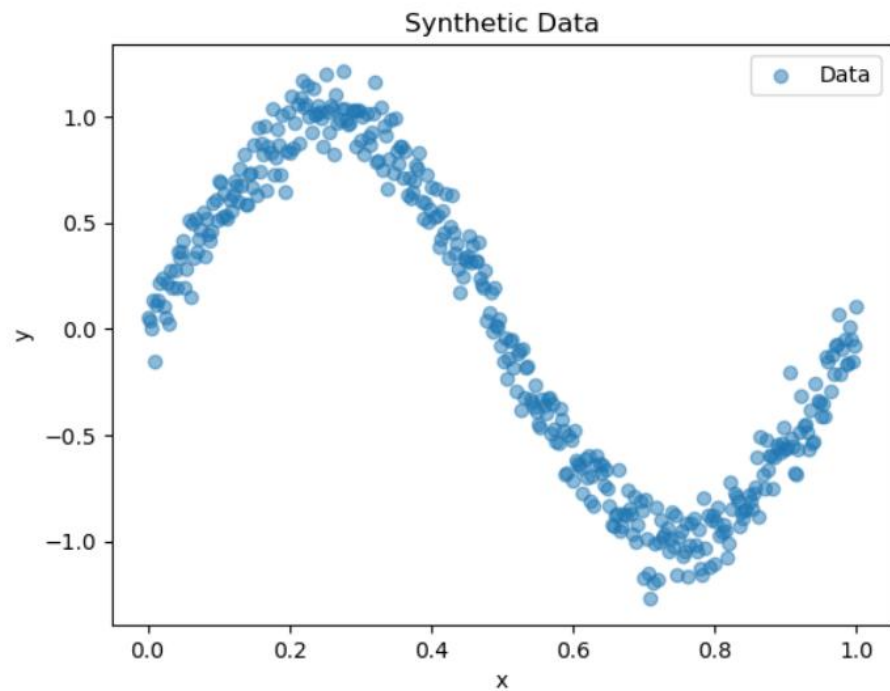


- Batch gradient descent
- Mini-batch gradient Descent
- Stochastic gradient descent

At each step of training:

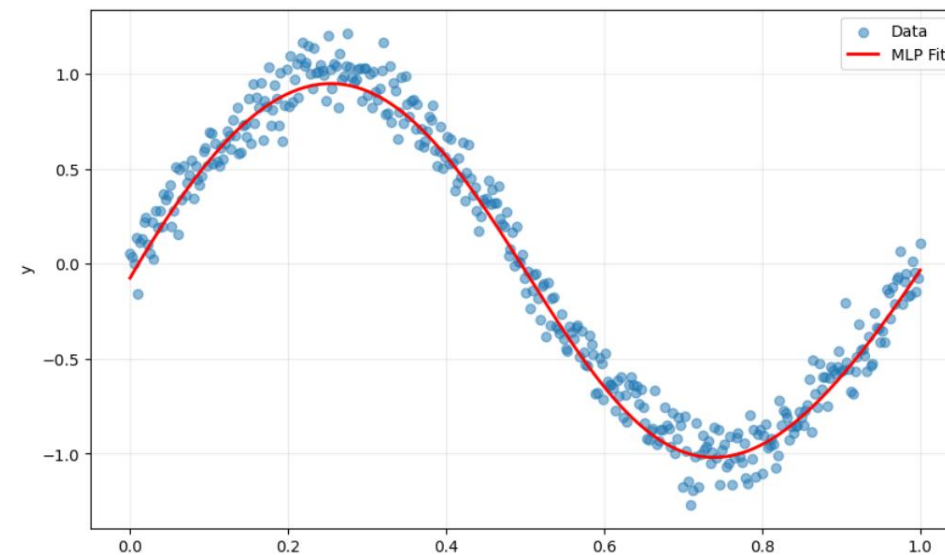
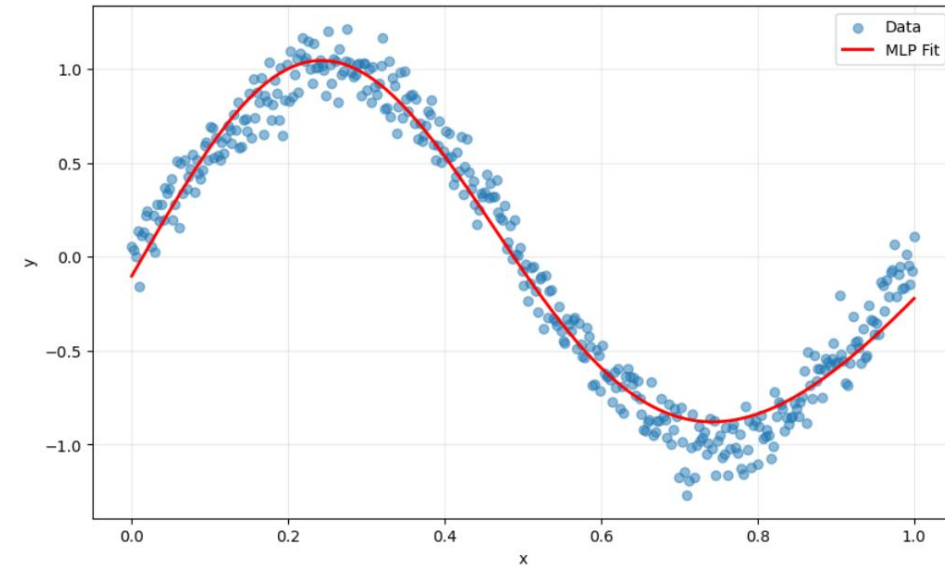
- Randomly select $N_{batchsize}$ datapoints from your dataset
- Perform a gradient descent update as usual

Today's exercise: why use minibatching?



Faster
Much smaller
computational graph

More accurate
Escape local minima



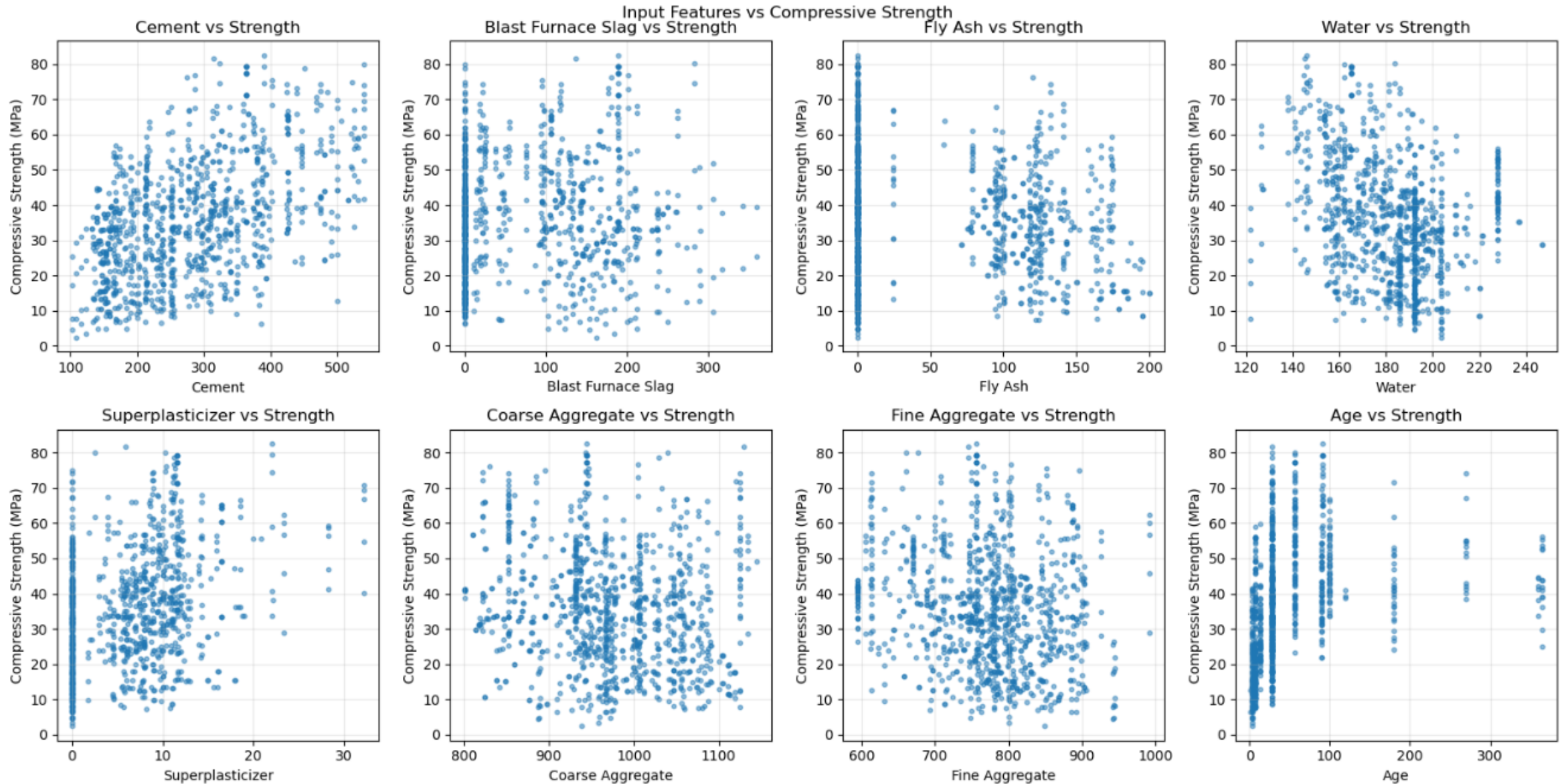
Today's exercise: Predicting performance of concrete

The dataset contains 8 input features that describe the composition and age of concrete samples:

1. **Cement (kg/m^3)** - Amount of Portland cement in the concrete mixture. Cement is the binding agent that holds the concrete together.
2. **Blast Furnace Slag (kg/m^3)** - A byproduct from iron production used as a supplementary cementitious material. It can partially replace cement and improve durability.
3. **Fly Ash (kg/m^3)** - Fine powder byproduct from coal combustion in power plants. Used as a partial cement replacement to improve workability and long-term strength.
4. **Water (kg/m^3)** - Amount of water in the mixture. The water-to-cement ratio is critical for strength - too much weakens the concrete, too little makes it unworkable.
5. **Superplasticizer (kg/m^3)** - Chemical additive that improves the flowability of concrete without adding extra water. Allows for easier pouring and better compaction.
6. **Coarse Aggregate (kg/m^3)** - Larger particles like gravel or crushed stone (typically $> 4.75\text{mm}$). Provides structural strength and bulk to the concrete.
7. **Fine Aggregate (kg/m^3)** - Smaller particles like sand (typically $< 4.75\text{mm}$). Fills voids between coarse aggregate and improves workability.
8. **Age (days)** - Time since the concrete was cast. Concrete continues to gain strength over time as the cement hydration process continues (typically measured at 1, 7, 28, or 90 days).

Target Variable: Compressive Strength (MPa) - The maximum stress the concrete can withstand before failing under compression, measured in megapascals.

First thing to do when you get a dataset – plot it



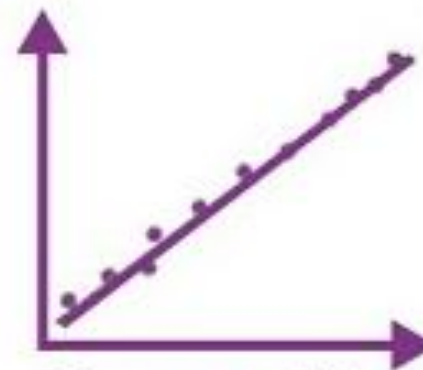
Second thing to do when you get a dataset – look for correlations

Pearson correlation coefficient

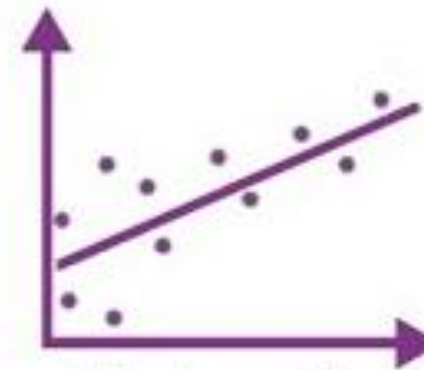
$$r_{xy} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}$$

Where:

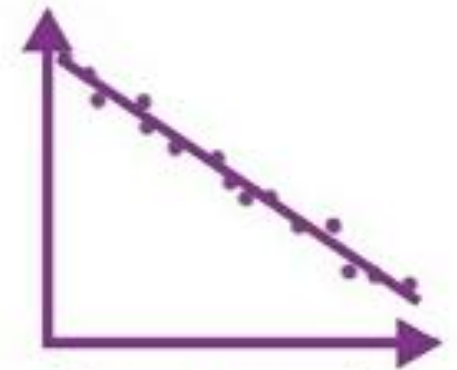
- x_i, y_i are individual data points
- $\bar{x}, \bar{y}$ are the means of x and y
- n is the number of data points



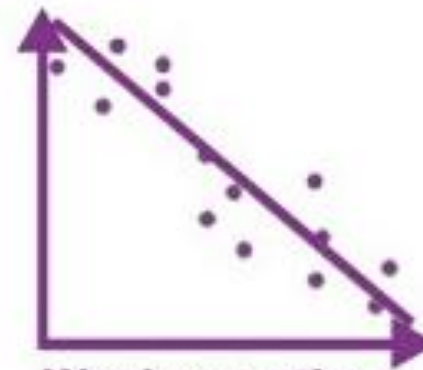
Strong positive correlation



Weak positive correlation



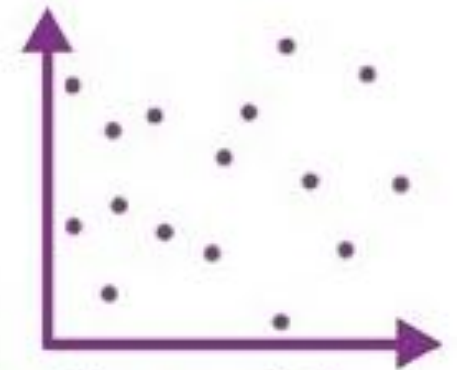
Strong negative correlation



Weak negative correlation



Moderate negative correlation



No correlation

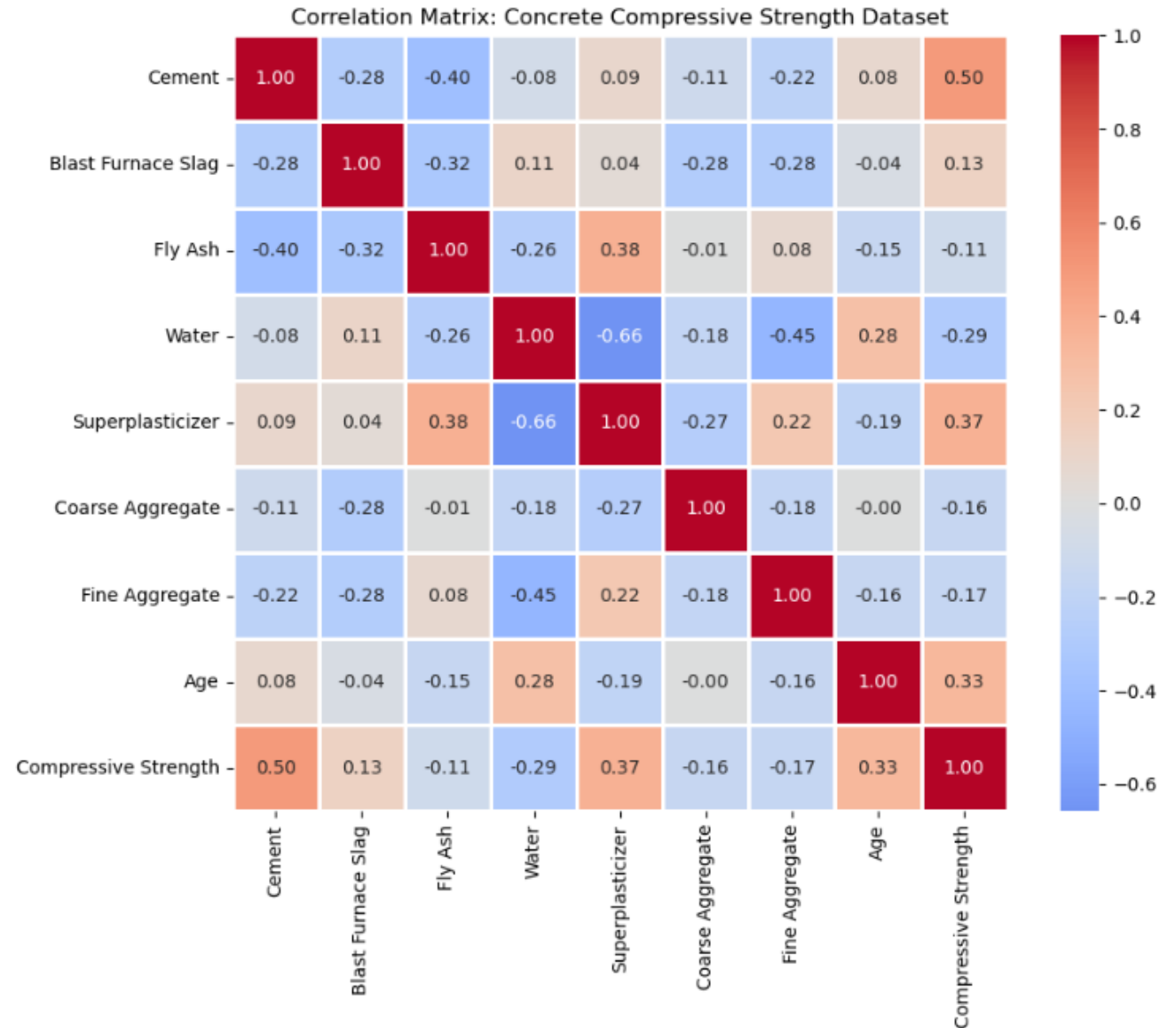
Second thing to do when you get a dataset – look for correlations

Pearson correlation coefficient

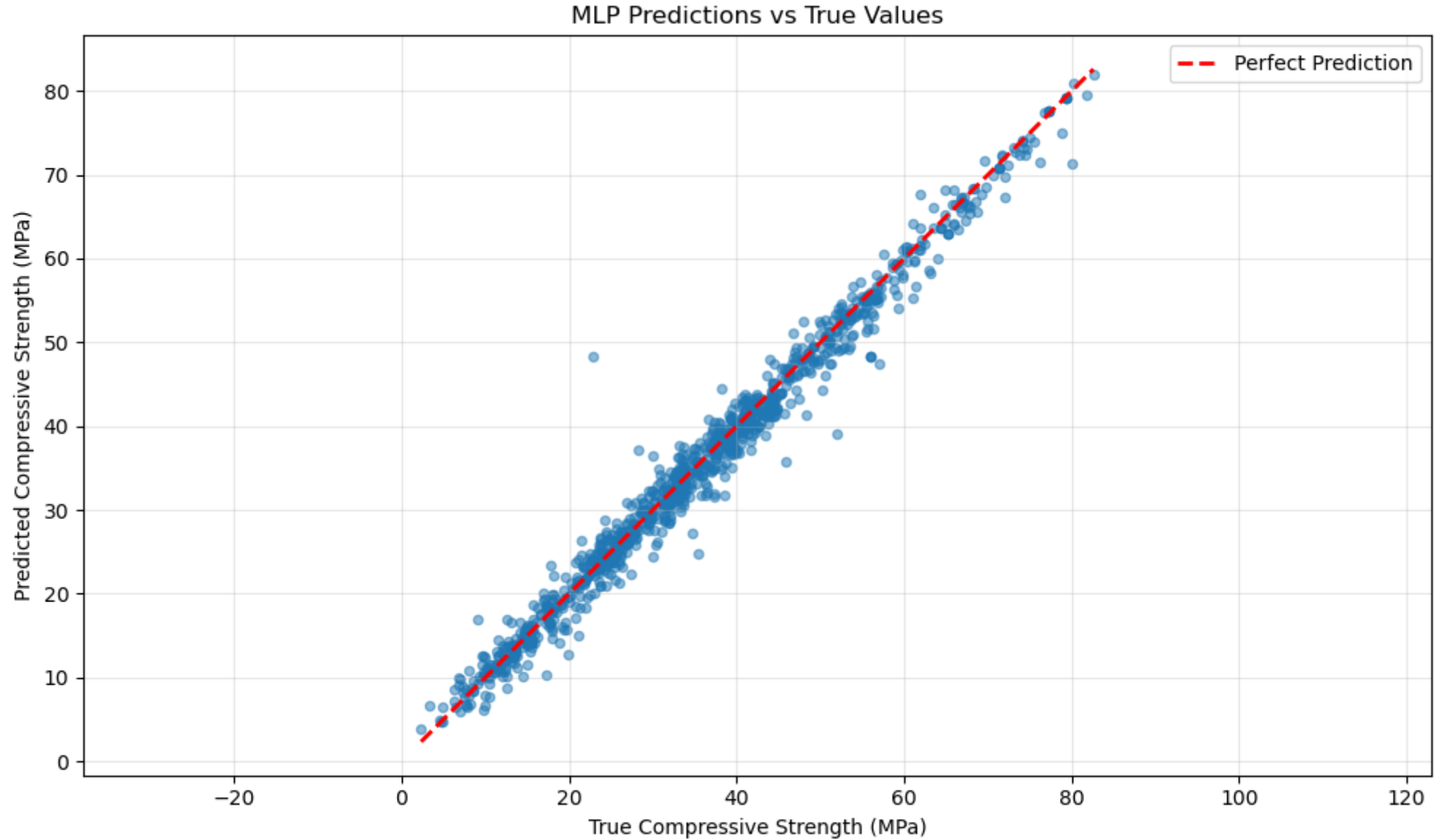
$$r_{xy} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}$$

Where:

- x_i, y_i are individual data points
- $\bar{x}, \bar{y}$ are the means of x and y
- n is the number of data points



Can a deep network sort out a nonlinear relationship from inputs to outputs?

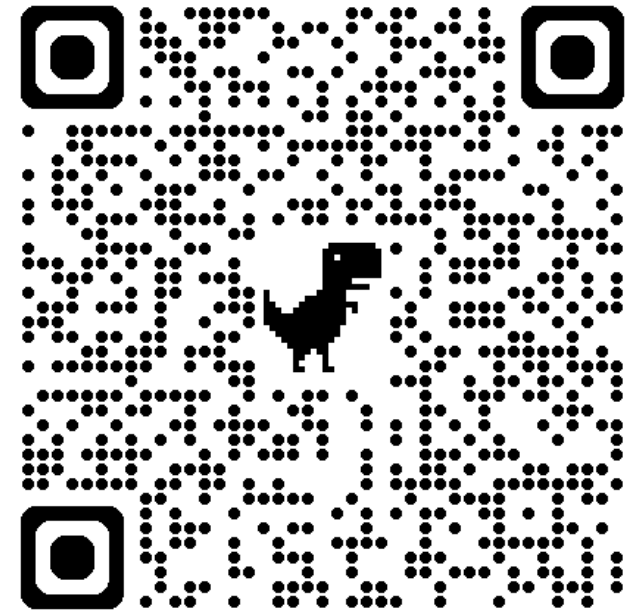


ENGR1050 Final Project Part Ordering F

If you intend to do a Pico based project, please fill out this form to order your parts. Orders must be received by the end of day Wednesday to receive on time.

- The budget per group is \$25/student
- Receiving parts on time will require providing a vendor with a fast delivery time. Digikey and Newark are preferred vendors.
- Before ordering a part, check the estimated delivery time is prior to Dec 3rd.
- Check with Dr. Trask (on Ed is fine) if you are getting something esoteric outside of what is available in class.
- You are responsible for doing research ahead of time to ensure you have appropriate skills to use the part (e.g. no complicated wiring diagrams, no soldering, no 3D printing etc)
- Orders that don't meet requirements will be rejected

To keep things simple, use low tech solutions where possible (e.g. rather than 3D printing a button mount, you can attach to a piece of cardboard). This will keep components secure (wiring and programming) and save you time.



Orders are coming in!

Keep an eye out for emails from me and Peter Bruno – you can pick them up from his office just outside the GM lab